CS5351 历年题解答（中英双语）

Past Quiz Answers · Bilingual · 覆盖 Week 1–6 考点

试卷来源：CS5351-202425 Quiz（开卷）

整合：Week 1–6 课件 + 六课连贯教学讲稿 + 已有双语笔记

Exam date: Oct 20 · 开卷考试

CS5351 软件工程期中历年题全解（题目 + 翻译 + 中英双语作答）

试卷：CS5351-202425 Quiz（开卷，覆盖第 1–6 课）结构：每题给出**题目原文（English original）→ 题目全译（中文）→ 双语解答（英文作答在前，中文对照在后）**。说明：题目原文已按试卷内容转录，仅清理了少量扫描/OCR杂讯；答案刻意避开课件原例。 Note: questions are transcribed faithfully with minor OCR cleanup; answers avoid course examples.

Question 1. Software Process (25%)

题目原文 / Original Question

A company founded by a group of fresh CS graduates uses the following software development process to develop their software products. You are one of the founders. You heard about the good things about agile processes and the need to handle technical debts with quality control through code review and the requirement engineering process should be rigorous. You would like to propose using them to develop the new project you receive. Nonetheless, the company has no prior experience in agile development.

[Scenario A] The current software process expressed by a project manager

Both my system analyst (SA) and I visit our client and meet with the client representatives to collect the software requirements through a series of meetings. The client elaborates the requirements and their purposes each meeting, and we listen. We ask questions to seek to confirm the requirements. Sometimes, they also provide us with sample inputs and outputs of their current systems for our reference. After each meeting, we prepare, update, and review a document named Document R to itemize the software requirements in textual forms. The outcome of the meeting series is to produce a finalized version of Document R, and the client signs off on this version (as a part of the contract and, as such, our company receives some money from the client). The user interfaces and how the user interfaces accept inputs and generate outputs are also in Document R.

I break down and re-group the requirements into subsystems and schedule my team to develop these subsystems one by one. I allocate the same amount of time to develop each subsystem to make the schedule predictable to the team and arrange the order of these subsystems into a schedule. I also append the schedule with the system integration and user acceptance test phases. The SA will refine my requirement allocations to subsystems and let me review and approve.

The programmers (Ps) study the requirements for each subsystem and discuss them with the SA in case of ambiguity. Sometimes, they also involve me in the discussion. If needed, the SA arranges additional meetings with the client representatives and discusses the details of the requirements listed in Document R for that subsystem. We will limit the number of such additional meetings to keep our company image professional.

We focus on delivering the requirements with the simplest possible design to gain speed and lower the development cost. Each subsystem is implemented by one P. The SA will set the priority of the tasks for the P to implement. The P also tests the subsystem manually while reading the requirements for the subsystem. The SA tests the subsystem implemented by the P. Of course, the SA also develops some code for the subsystem.

If there is any delay in the schedule for a subsystem, we work overtime on weekdays and weekends. We cut the schedule for the next subsystem if the delay of the current subsystem is serious to meet the original project milestones.

After producing all the subsystems, the SA and the Ps will integrate them into a whole system and test it. They test those scenarios related to what they have tested in individual subsystems and brainstorm new scenarios to test the whole system. When developing individual subsystems or integrating the whole system, critical and major bugs are fixed if found. Integrating the whole system is challenging as many unexpected bugs usually pop up at this stage.

When we judge the system is ready, we fix all these minor bugs the clients may observe and arrange the clients to conduct a user acceptance test. Any bug found by the client and within the scope of the contract is fixed by us. After passing the user acceptance test, we write the documentation as listed in the contract previously signed by the client. If the source code is also required to be handed over to the client, we will delete all these debugging statements from the source code and pass the resulting code base to the client. Deferring the documentation and minor bug fixing to the final stage helps my team reduce the overheads in code and document revisions. Finally, we pass the system to the client.

Your goal is to explain how an agile process differs from the current process (i.e., Scenario A) to your company partners. Your explanation should be supported by the information of the current process (e.g., a concrete example or description of the current process). **(5 points for each)**

(a) The organization of the development phases in the software development lifecycle

(b) The involvement of customers

(c) Prioritization of tasks to perform

(d) The duration period of a subsystem

(e) Programmers' control on the tasks they perform

题目全译 / Chinese Translation

一家由应届计算机毕业生创立的公司用下列软件开发流程开发产品。你是创始人之一。你听说了敏捷过程的种种好处，也听说需要通过代码审查进行质量控制来处理技术债务，需求工程过程也应当严谨，因此想在新项目中推行这些方法。但公司此前没有敏捷开发经验。

【场景 A】项目经理描述的现行软件过程

我和系统分析员（SA）拜访客户，与客户代表开一系列会议收集软件需求。客户每次会议阐述需求及其目的，我们倾听，并通过提问确认需求。有时他们也提供现有系统的样例输入输出供我们参考。每次会后，我们编写、更新和评审一份名为"文档 R"的文件，逐条列出文字形式的需求。这一系列会议的最终成果是文档 R 的定稿，客户签署确认（作为合同的一部分，公司因此从客户处获得款项）。用户界面以及界面如何接受输入、产生输出，也都写在文档 R 中。

我把需求拆分重组为若干子系统，安排团队**逐个**开发。我给**每个子系统分配相同的时间，**使排期对团队可预测，并把子系统的先后顺序排入进度表，表后附上系统集成和用户验收测试阶段。SA会细化我的需求到子系统的分配，交我评审批准。

程序员（P）研读每个子系统的需求，有歧义时与 SA 讨论，有时也拉我一起讨论。必要时 SA会约客户代表开追加会议，讨论文档 R 中该子系统的需求细节。但我们会**限制这类追加会议的次数，**以保持公司形象专业。

我们用最简单的可能设计来交付需求，以换取速度、降低开发成本。**每个子系统由一名 P 实现。任务优先级由 SA 来定。** P 一边读需求一边手工测试子系统；SA 测试 P 实现的子系统（SA 也写一部分代码）。

如果某个子系统进度延误，我们工作日和周末都加班；若延误严重，就**压缩下一个子系统的工期，**以保证原定项目里程碑。

所有子系统做完之后，SA 和 P 把它们集成为整个系统并测试：重测各子系统已测过的场景，再头脑风暴出新场景测全系统。无论开发子系统还是集成阶段，发现严重和主要缺陷就修。集成整个系统很吃力，因为这阶段常会冒出许多意外 bug。

我们判断系统就绪后，修掉客户可能看到的所有次要缺陷，安排客户做用户验收测试。客户在合同范围内发现的任何缺陷由我们修复。通过验收后，按先前签署的合同清单编写文档。若需向客户交付源代码，我们会删除所有调试语句后交出代码库。**把文档编写和次要缺陷修复推迟到最后阶段，有助于团队减少代码与文档修订的开销。**
最后把系统交给客户。

你的目标是向合伙人解释**敏捷过程与现行过程（场景 A）有何不同。**解释必须有现行过程的信息支撑（例如现行过程的具体例子或描述）。**（每小题 5 分）**

(a) 软件开发生命周期中各开发阶段的**组织方式**

(b) 客户的**参与程度**

(c) 任务的**优先级排序**

(d) 一个子系统的**开发周期长短**

(e) 程序员对自己所执行任务的**控制权**

解答 / Answers

(a) The organization of the development phases 开发阶段的组织方式

English answer:

In an agile process, all development activities (requirements, design, coding, testing) are performed **iteratively and in parallel within each short iteration**, and each iteration delivers a small but working, potentially shippable increment. In Scenario A, the lifecycle is strictly **sequential and phase-based**: requirements are frozen first (Document R is signed off as part of the contract), then subsystems are designed and implemented **one by one**, and integration and testing happen **only after all subsystems are produced** — the scenario explicitly says "After producing all the subsystems, the SA and the Ps will integrate them into a whole system," and documentation is deferred to the final stage. Agile would instead integrate and test continuously in every iteration, so that "many unexpected bugs popping up at integration" are discovered early in small batches.

中文对照： 敏捷过程中，需求、设计、编码、测试在**每个短迭代内并行、迭代地进行，**每次迭代都交付一个可运行的小增量。而场景 A 是**严格按阶段顺序**推进：先把需求冻结（文档 R 签约），然后**逐个**开发子系统，**全部做完之后**才统一集成测试——文中明确说"After producing all the subsystems...integrate them"，文档也推迟到最后。敏捷会在每次迭代中持续集成和测试，从而避免大规模集成时"意外 bug 爆发"。

(b) The involvement of customers 客户的参与程度

English answer:

Agile requires **continuous customer involvement** throughout the whole project (e.g., XP's on-site customer, or a Sprint Review with the customer at the end of every sprint); the customer can change their mind at any time and feedback is gathered every iteration. In Scenario A, the client is involved only **at the very beginning** (a series of meetings to collect requirements, then signing off Document R) and **at the very end** (user acceptance test). Worse, the team "limits the number of additional meetings to keep the company image professional," which actively blocks mid-project customer feedback — so misunderstandings buried in Document R can only be discovered very late.

中文对照： 敏捷要求客户**全程持续参与**（如 XP 的现场客户、每个 Sprint的评审会），客户可以随时改变主意。而场景 A 中客户只在**项目开头**（开需求会、签文档 R）和**结尾**（用户验收测试）出现；团队甚至"为了保持专业形象而限制追加会议次数"，主动切断了开发过程中的客户反馈，导致需求误解要到很晚才暴露。

(c) Prioritization of tasks to perform 任务的优先级排序

English answer:

In agile, tasks are prioritized by **business value**: the Product Owner orders the product backlog so that the most valuable user stories are implemented first, and priorities can be re-ordered every sprint. In Scenario A, prioritization is done **top-down by the SA and the PM** ("The SA will set the priority of the tasks for the P to implement"), and the PM allocates **the same amount of time to each subsystem** and arranges them into a fixed schedule. This means high-value and low-value subsystems get identical resources, and priorities cannot respond to changing business needs.

中文对照： 敏捷按**业务价值**排序——Product Owner 排列产品待办列表，最有价值的用户故事先做，且每个 Sprint 都可重排。场景 A 中则由 **SA 和项目经理自上而下**定优先级（"The SA will set the priority of the tasks"），并给**每个子系统分配相同的时间、**按固定顺序排期——高价值和低价值子系统得到同样资源，优先级无法响应业务变化。

(d) The duration period of a subsystem 子系统的开发周期

English answer:

Agile works in **short, fixed-length iterations** (e.g., 1–4 week sprints or monthly XP releases), each ending with a reviewable increment; the iteration length is fixed and the scope is adjusted to fit it. In Scenario A, each subsystem is given **one equal, arbitrarily allocated block of time** to "make the schedule predictable," and when a delay occurs the team works overtime and **cuts the schedule of the next subsystem** to protect the original milestones — i.e., time is fixed by decree and quality/scope silently sacrificed, rather than adjusting scope transparently at short, regular intervals.

中文对照： 敏捷采用**短而固定长度的迭代**（如 1–4 周的 Sprint、XP每月发布），迭代长度固定、通过调整范围来适配。场景 A 中每个子系统被分配**一块均等、随意设定的时间**以保证"排期可预测"；一旦延期就加班，甚至**压缩下一个子系统的工期**来保护原有里程碑——即靠行政命令固定时间，悄悄牺牲质量和范围，而不是在规律的短周期内透明地调整范围。

(e) Programmers' control on the tasks they perform 程序员对任务的控制权

English answer:

Agile teams are **self-organizing**: developers pull tasks from the sprint backlog themselves, decide **how** to implement them, and practices such as pair programming and collective ownership give programmers real control and shared responsibility. In Scenario A, programmers have almost no control: each P is **assigned** a subsystem by the PM/SA, implements it alone ("Each subsystem is implemented by one P"), the **SA sets the task priorities**, overtime is imposed when the schedule slips, and even testing follows a script decided by others. Programmers are executors of a plan, not owners of their work.

中文对照： 敏捷团队**自组织：**开发者自己从 Sprint待办列表领取任务，自己决定**怎么做，**结对编程、集体所有权让程序员真正掌控并共担责任。场景 A中程序员几乎没有控制权：每个 P 由 PM/SA **分派**一个子系统、**独自**实现，**优先级由 SA 定，**延期就强制加班，测试也按别人的安排做。程序员只是计划的执行者，而不是工作的主人。

Question 2. Software Requirements (25%)

题目原文 / Original Question

(a) Give one example of a measurable non-functional requirement and describe what changes to your example so that it becomes non-measurable. Explain your change. You are not allowed to take any examples from the materials of this course. **(5 points)**

(b) Requirement elicitation has five basic types of activities. How do you use them to improve the requirement elicitation activities described in the current software process in Question 1? Your answer should be explicitly and directly relevant to the information described in Scenario A. **(20 points, one area of improvement carries 4 points)**

- General/generic answers or vague answers may receive zero marks.

题目全译 / Chinese Translation

(a) 给出一个**可测量的非功能需求**示例，并说明对你的示例做什么修改会使它变成**不可测量**的。解释你的修改。不得使用本课程材料中的任何示例。（5 分）

(b) 需求获取有**五种基本类型的活动。**你如何用它们来改进 Question 1中现行软件过程所描述的需求获取活动？你的答案必须与场景 A 中描述的信息**明确、直接相关。**（20 分，每个改进点4 分）

- 笼统/泛泛或含糊的回答可能得零分。

解答 / Answers

(a) Measurable → non-measurable non-functional requirement (5 pts)

English answer:

Example of a measurable non-functional requirement:

"The mobile app shall complete user login and display the home page within 2 seconds, measured on a mid-range phone over a 4G network, in 95% of attempts during peak hours."

Change made and explanation:

Remove all the numbers and contexts, making it: "The app shall log users in quickly." This becomes **non-measurable** because "quickly" is subjective — there is no metric (no time threshold), no measurement condition (device, network, percentile), and no way to design a test that decides pass or fail. As stated in the course, if we do not know how far the current system is from the goal, we cannot design the system to meet the goal; an unverifiable requirement is effectively no requirement at all.

中文对照：

可测量的非功能需求示例：

"在中端手机、4G 网络环境下，高峰时段 95% 的尝试中，App 应在 2 秒内完成登录并显示首页。"

修改及解释：

把数字和条件全部去掉，变成"App 应快速完成登录"。这样就**不可测量**了——"快速"是主观词，没有度量指标（时间阈值）、没有测量条件（设备、网络、百分位），无法设计判定通过/失败的测试。正如课程所讲：不知道当前系统离目标有多远，就无法设计系统去达成目标；不可验证的需求等于没有需求。

> 注意：本答案刻意避开了课件中的例子（加密货币价格 1 秒内显示、PS5 购买等待 30 分钟、99.9% 可用性等）。

(b) Improving Scenario A's elicitation with the five basic activities (20 pts)

需求获取五活动 / The five basic elicitation activities: **(1) understand the application domain, (2) identify requirement sources, (3) analyze stakeholders, (4) select elicitation techniques, methods and tools, (5) elicit requirements from stakeholders and other sources**（迭代进行，不能做完一个再开始下一个）。

English answer:

① **Understand the application domain.** Currently the PM and SA only "visit the client and listen" in meetings, without studying the client's business first. Improvement: before and between meetings, the SA should thoroughly explore the client's current environment — the business processes behind the sample inputs/outputs the client provided, plus the organizational, political and social constraints — so that questions asked in meetings are informed and Document R reflects the real domain rather than only what clients happen to say.

② **Identify requirement sources.** Currently the only source is "the client representatives" in meetings. Improvement: the team should identify **all** sources — the end users who will actually operate the system (not just their representatives), domain experts, the client's existing systems and their manuals, forms and reports, and the sample inputs/outputs already provided — because requirements are scattered across many sources in different formats.

③ **Analyze stakeholders.** Currently no stakeholder analysis is done; whoever shows up at the meeting speaks for everyone. Improvement: systematically identify internal and external stakeholders, find **key user representatives and product champions** who support the software, and consult them during elicitation, so that requirements capture the needs of all user groups instead of only the loudest client representatives.

④ **Select techniques, methods and tools.** Currently the only technique is a series of question-and-confirm meetings. Improvement: use a **variety of techniques in combination** — since the client already provides sample inputs/outputs, **task analysis** and **scenario analysis** can be applied to them; since user interfaces are part of Document R, build **prototypes** of the UIs for clients to react to; use **brainstorming / focus groups /**

facilitated workshops for ambiguous areas. This matches the course point that elicitation is best performed with multiple techniques.

⑤ **Elicit requirements from stakeholders.** Currently elicitation ends once Document R is signed, and further meetings are deliberately capped "to keep the company image professional." Improvement: elicit requirements **iteratively throughout the project** — establish the system scope early, investigate stakeholders' needs in detail, define the future business processes the system must support, and check them against the client's business goals — because elicitation is an iterative process and requirements will evolve while subsystems are being designed.

中文对照：

① **理解应用领域。** 现状是 PM 和 SA 只是去客户那里"听"，事前没有研究客户的业务。改进：会前与会间，SA 应彻底探索客户的现行环境——客户提供的样例输入输出背后的业务流程，以及组织、政治、社会约束——让会议提问有据可依，文档 R 反映真实领域而不只是客户口头所言。

② **识别需求来源。** 现状是唯一来源是"客户代表"。改进：识别**全部**来源——真正操作系统的最终用户（而非仅代表）、领域专家、客户现有系统及其手册/表格/报告、已提供的样例输入输出。需求分散在众多来源中，格式各异。

③ **分析利益相关者。** 现状是没有做利益相关者分析，谁出席会议谁代言。改进：系统识别内外部利益相关者，找出**关键用户代表和产品冠军（product champions）**并在获取过程中咨询他们，让需求覆盖所有用户群体，而非只听最响亮的客户代表。

④ **选择技术、方法和工具。** 现状是唯一技术是"提问—确认"式系列会议。改进：**组合多种技术**——客户已提供样例输入输出，可做**任务分析、场景分析；**用户界面也写入文档 R，可为 UI 做**原型**让客户反馈；对模糊领域用**头脑风暴/焦点小组/引导式研讨会。**呼应课程观点：获取最好使用多种技术组合。

⑤ **从利益相关者获取需求。** 现状是文档 R 一签，获取即告结束，还故意限制追加会议。改进：**在整个项目中迭代地获取**——尽早确立系统范围、详细调查利益相关者需求、定义系统要支撑的未来业务流程、对照客户业务目标检验——因为获取是迭代过程，子系统设计过程中需求会继续演化。

Question 3. Software Architecture (25%)

题目原文 / Original Question

LeaveHomeSafe (Chinese: 安心出行) is a digital contact tracing mobile application provided by the Hong Kong Government during the COVID time. It has a few functionalities:

- scan a QR code and press a button to indicate that the user has entered a shop
- press a button to indicate the user leaves the shop
- automatically leave a shop after a certain time (e.g., an hour)
- receive a notification over the internet and notify whether the shop entered in the past N days incurs any COVID-19 threat.

There was a comment that the app should be enhanced to ease the visually impaired persons (e.g., a person with a decreased ability to see) to use the app. For instance, such a person may be unable to locate and scan a QR code with ease.

The following is a list of requirements:

(a) Each shop should be assigned with a unique beep sequence.

(b) Each shop should install a single speaker to emit its beep sequence repeatedly when the speaker turns on.

(c) A beep can be either short or long like the Morse code. The length of the beep sequence for a shop should not be longer than 32 beeps, and the sound emitted by the speaker should be within the range suitable for the population of visually impaired persons.

An AI chatbot provides the following information: some studies suggest that blind individuals may be better at distinguishing between similar frequencies or pitches, particularly in the higher frequency range (above 2,000 Hz); when designing a beep sequence that is more likely to be perceived by blind individuals without being overly disruptive to sighted individuals, consider: (1) frequency range 1,000–4,000 Hz; (2) slight pitch variation (~100 Hz) between beeps; (3) clear rhythm and duration, short distinct beeps of 200–300 ms with pauses; (4) moderate volume; (5) harmonic or melodic sequences rather than pure tones.

(d) The app can hear a beep sequence from a speaker with an accuracy of 95% or higher from the microphone of a low-end mobile phone.

(e) The app can use the beeps heard to identify a shop with an accuracy of 95% or higher.

(f) Within 3 seconds after the app speaks out the shop name, if a user uses a figure to knock on the phone display three times within 1.5 seconds, the app will mark the user to enter the shop.

(g) The app will cache the mapping between the beep sequence heard in item (e) and the marked shop in item (f) for one month to speed up the next round of processing of the shop.

(a) Extract four quality attributes of the app from the requirement and paraphrase them into your own words. For each quality attribute, you should also state the suitability of Model, View, and Controller (MVC) architecture. **(20 points)** — Note that direct copying will receive no mark.

(b) The target user population are visually impaired persons. The person is difficult to know if the enhanced app is not responding or crashed. Sketch a design plan to design the app to address this issue. **(5 points)**

题目全译 / Chinese Translation

「安心出行」是香港政府在新冠期间提供的数字接触追踪手机应用，功能包括：扫码并按键标记入店；按键标记离店；一段时间后（如一小时）自动离店；通过互联网接收通知，提示过去 N 天内进过的店铺是否有新冠风险。

有人评论说，应增强该应用以方便**视障人士**（视力低下者）使用——例如这类人难以找到并扫描二维码。

需求清单：

(a) 每个店铺应被分配**唯一**的蜂鸣序列。

(b) 每个店铺应安装**一个**扬声器，开启后**循环**播放其蜂鸣序列。

(c) 蜂鸣可长可短（类似摩斯电码）；店铺的蜂鸣序列长度**不超过 32 个蜂鸣**；声音频率应落在**适合视障人群**的范围内。

（AI 聊天机器人为 (c) 提供了参考信息：盲人可能对相近频率/音高更敏感，尤其在 2,000 Hz 以上；设计建议：①频率 1,000–4,000 Hz；②蜂鸣间轻微变调约 100 Hz；③节奏清晰，短蜂鸣 200–300 毫秒并带停顿；④音量适中；⑤用谐波/旋律序列而非纯音。）

(d) 应用应能通过**低端手机**的麦克风，以 **95% 或更高**的准确率听到扬声器发出的蜂鸣序列。

(e) 应用应能利用听到的蜂鸣，以 **95% 或更高**的准确率**识别出店铺**。

(f) 在应用**报出店名后 3 秒内**，若用户在 **1.5 秒内**用手指敲击手机屏幕**三次**，应用将标记用户**进入**该店铺。

(g) 应用将把 (e) 中听到的蜂鸣序列与 (f) 中标记的店铺之间的映射**缓存一个月**，以加速下一轮处理。

问题 (a)： 从需求中提取该应用的**四个质量属性**，并**用自己的话转述**（照抄不得分）。对每个质量属性，还须说明 **MVC 架构是否适用**。（20 分）

问题 (b)：
目标用户是视障人士，他们**难以知道增强后的应用是没有响应还是已经崩溃**。勾画一个设计方案来解决此问题。（5 分）

解答 / Answers

(a) Four quality attributes + suitability of MVC (20 pts)

English answer:

QA 1 — Recognition accuracy under low-end hardware (from requirement (d)).

Requirement (d) demands the app hear a shop's beep sequence with **⩾95% accuracy from the microphone of a low-end phone**. Paraphrased: audio-sensing accuracy / dependability in a resource-constrained environment.

MVC suitability: **Suitable, but MVC alone does not guarantee it.** The audio-capture and decoding logic can be encapsulated in the **Model**, separated from UI concerns, so it can be developed and tested in isolation; however, achieving 95% accuracy mainly depends on the signal-processing design (e.g., the frequency/rhythm guidance in requirement (c)), not on the architectural style itself.

QA 2 — Shop identification correctness (from requirement (e)).

Requirement (e) demands the app identify the shop from the heard beeps with **⩾95% accuracy**. Paraphrased: correctness / reliability of the core identification function.

MVC suitability: **Suitable.** The beep-to-shop mapping and matching logic naturally live in the **Model**; the View and Controller do not affect the result. MVC's separation makes the identification logic independently testable against the 95% target, and requirement (g)'s cache can also be hidden inside the Model.

QA 3 — Responsiveness to user gestures (from requirement (f)).

Requirement (f) specifies tight timing: after the app speaks the shop name, three knocks within 1.5 seconds (within a 3-second window) must mark the user as entered. Paraphrased: responsiveness / latency — which is simultaneously a usability concern, because users are visually impaired and cannot see progress indicators.

MVC suitability: **Suitable.** The **Controller** is precisely the component designed to accept user input (the knocking gesture) and update the Model quickly, while the **View** gives immediate feedback; separating View from Model prevents slow rendering from blocking input handling.

QA 4 — Processing efficiency / performance (from requirement (g)).

Requirement (g) requires caching the beep-sequence-to-shop mapping for one month "to speed up the next round of processing." Paraphrased: efficiency / performance optimization through caching.

MVC suitability: **Suitable.** Caching is an internal Model concern: the Model can be optimized (a cache layer) without any change to the View or Controller, and multiple Views (e.g., a dedicated UI for visually impaired users) automatically benefit.

(Alternative fourth attribute — Accessibility / audibility for the target population (from (c)): the beep must lie in a frequency range perceivable by visually impaired persons, with sequences no longer than 32 beeps. Paraphrased: *accessibility (audibility) of the acoustic channel for the target user group. MVC suitability: **Suitable** — the **View** can be specialized for visually impaired users — audio feedback, large fonts, vibration — while the Model stays unchanged; this is exactly the benefit of separating View from Model.)*

Grading note: any four of the five attributes above are acceptable; what matters is (i) paraphrasing instead of copying, and (ii) a justified suitability statement for each.

中文对照：

质量属性 1——低端硬件下的识别准确率（源自需求 (d)）。 需求 (d) 要求应用用低端手机的麦克风以 **⩾95% 的准确率**听到店铺的蜂鸣序列。转述：资源受限环境下的音频感知准确性/可靠性。 MVC 适用性：**适用，但单靠 MVC无法保证**。音频采集与解码逻辑可封装在 **Model** 中、与 UI 分离，便于独立开发和测试；但 95% 准确率主要取决于信号处理设计（如需求 (c) 的频率/节奏建议），而非架构风格本身。

质量属性 2——店铺识别正确性（源自需求 (e)）。 需求 (e) 要求根据听到的蜂鸣**以 ⩾95% 的准确率识别出店铺**。转述：核心识别功能的正确性/可靠性。 MVC 适用性：**适用**。蜂鸣—店铺映射与匹配逻辑天然属于 **Model**，View 和 Controller 不影响结果；分离使识别逻辑可针对95% 目标独立测试，需求 (g) 的缓存也可藏在 Model 内部。

质量属性 3——对用户手势的响应性（源自需求 (f)）。 需求 (f) 规定严格时序：报出店名后 3 秒内，用户 1.5 秒内敲屏三次即标记入店。转述：响应性/延迟（同时关乎可用性——用户是视障人士，看不见进度指示）。

MVC 适用性：**适用。Controller** 正是接收用户输入（敲击手势）并快速更新 Model 的组件，**View** 可立即给出反馈；View与 Model 分离可避免慢渲染阻塞输入处理。

质量属性 4——处理效率/性能（源自需求 (g)）。 需求 (g)
要求把蜂鸣序列与店铺映射缓存一个月，"以加速下一轮处理"。转述：通过缓存实现的效率/性能优化。
MVC 适用性：**适用。**缓存是 Model 的内部事务：Model 可自行优化（加缓存层）而 View、Controller 完全不用改，多个View（如视障专用界面）自动受益。

（备选第四个属性——目标人群的可听性/无障碍（源自 (c)）：蜂鸣频率须落在视障人群可感知的范围内，序列 ≤32 个蜂鸣。MVC 适用性：**适用——View** 可专为视障用户定制（语音反馈、大字、振动），Model 不变，这正是 View/Model 分离的好处。）

评分提示：以上五个属性任选四个即可；关键是 (i) 用自己的话转述而非照抄，(ii) 每个属性都要有理由地判断 MVC 是否适用。

(b) Design plan for non-responding/crashed detection (5 pts)

English answer:

Problem: visually impaired users cannot see whether the app is busy, frozen, or crashed; a silent failure is indistinguishable from normal processing.

Design plan (a feedback-first, non-visual loop):

1. **Acknowledge every input immediately (perception).** Every user action — a button press or a knock — instantly produces a non-visual confirmation: a short, distinct tone or a vibration pattern. This gives users confidence the input was received.
2. **Announce state transitions by voice (a TTS status channel).** A text-to-speech engine audibly narrates the app's state machine: "listening…", "shop X entered, please knock to confirm", "processing, please wait". Requirement (f) already has the app speaking the shop name, so this extends an existing channel.
3. **Make failure detectable and recoverable (fault handling).** A watchdog monitors the processing pipeline; if no progress is made within a time limit, the app plays a distinct error sound, announces the failure ("app restarted, please knock again"), automatically restarts, and **restores its state from the persistent cache in requirement (g)**, so no entry/exit record is lost.
4. **Validate with real users.** Prototype the audio feedback with visually impaired users and iterate using scenario-based evaluation, since they are the population whose needs define success.

中文对照： 视障用户看不到应用是在忙、卡住还是崩溃，"无声的失败"和"正常处理"无法区分。**设计方案（以非视觉反馈为先的闭环）：**

1. **即时确认每次输入（感知层）。**
   每个用户动作（按键、敲击）都立刻产生非视觉确认音或振动，让用户确信输入已被接收。
2. **语音播报状态转换（TTS 状态通道）。** 用语音引擎播报应用状态机："正在收听……"、"已进入店铺 X，请敲击确认"、"处理中，请稍候"。需求 (f) 中应用本就要报出店名，顺势扩展即可。
3. **让故障可察觉、可恢复（容错层）。**
   看门狗监控处理流水线，超时无进展则播放独特的错误音并语音提示"已重启，请重新敲击"，自动重启并利用需求 (g) 的持久缓存恢复状态，进出店记录不丢失。
4. **与真实用户验证。** 用视障用户做原型场景测试并迭代，因为他们才是需求成功与否的最终裁判。

Question 4. Technical Debt (25%)

题目原文 / Original Question

(a) Technical debts can be classified into four dimensions. Determine the four dimensions of each of the following items. **(14 points)**

Dimensions: **Type** (D=design/architecture, C=code, T=test, O=documentation) · **Intentionality** (I=intentional, U=unintentional) · **Time horizon** (S=short-term, L=long-term) · **Degree of focus** (F=focused, U=unfocused). Example given in the paper: "Let's use the latest programming languages, tools, and frameworks to implement individual features in our project" = **C I L U**.

1. Blockchain is not ready yet. Our application does not need to support blockchain as a platform in the next two releases. Let us now build our system without the need to prepare it to be a decentralized application. Nonetheless, we should think about the consequence when we code our program if blockchain is coming back and we need to support it, and record our consideration into a file folder for later reference.
2. Make a change on a code file and forget to update the corresponding unit testing test cases. As such, we also forget to pass the test case code to our development buddies for code review.
3. Add hard-coded values and brute-force logic to bypass a hard-to-debug bug in a code region.
4. Developer A says, "You should follow the standard in writing your code." Developer B replies, "I followed. See?" Developer A finally finds out that the "standard" mentioned by Developer B is from his practice in B's former company. Developer A then asks Developer B to fix them by passing the standard of "our" company for Developer B to follow. Nonetheless, Developer B is busy with other coding activities and deferred the change of the wording in this piece of code to the next development cycle (sprint in Scrum for example).
5. We are running out of time, so everyone should be a programmer this week. Josh stop proofreading the documentation and help write the code with Mary.
6. In a review meeting on a piece of code, a developer says, "What is the purpose of this value here?" Another developer replies, "Who knows? It was there for years! The system works fine." After tracking the history, there were attempts to remove this value from the code, but it always breaks the code if removed. There was no idea on how to fix this issue.
7. Mary is competent. She is asked to improve the error handling of an application component. Mary is confident in completing the task in a week. However, as she digs into the code, she finds many inconsistencies in logic, bad dependencies, and other code issues. So, Mary decides to clean up all these issues before working on improving the error-handling part of the component. As she makes necessary changes, she further notices that there is no unit test case. She continues to make major changes to the code despite lacking unit test cases to assist her and finally commits the code to the main code repository. Everything looks alright. Nonetheless, she does not realize some deeply hidden dependency issues in some configuration files. The logic tied to some ad hoc values in some other code sections to bypass some previous bugs is now broken by her changes, making the application fail. Finally, she gets everything working, but the originally estimated one-week effort now becomes three weeks. Also, after she publishes (pushes) her changes, other developers sometimes can and sometimes cannot test the application due to the broken dependency. All developers are now puzzled. They do not know how to fix the broken problem while supporting the delivery and maintenance of the application.

(b) A saying is that accumulating technical debts will slowly kill a project and even a company. Many people say that technical debts are unavoidable. Many people also say that a project does not resolve all technical debts before the application retires. Propose a cost-effective strategy to handle technical debts in a software project. **(8 points)**

(c) Give one example of technical debt in your course project, which should belong to Documentation, Intentional, Short-term, and Unfocused. **(3 points)**

题目全译 / Chinese Translation

(a) 技术债务可分为四个维度。判定下列各项的四个维度。（14 分）

维度：**类型**（D=设计/架构，C=代码，T=测试，O=文档）·**意图性**（I=有意，U=无意）·**时间范围**（S=短期，L=长期）·**聚焦度**（F=聚焦，U=不聚焦）。卷面示例："让我们用最新的编程语言、工具和框架来实现项目中的各个功能" = **C I L U**。

1. 区块链技术尚不成熟。我们的应用在接下来的两个发布版本中不需要支持区块链平台。现在就构建系统，不必为成为去中心化应用做准备。但编程时要考虑"如果区块链卷土重来、我们需要支持它"的后果，并把我们的考虑记入一个文件目录以供日后参考。
2. 修改了一个代码文件，却忘记更新相应的单元测试用例；也因此忘记把测试用例代码交给开发伙伴做代码评审。
3. 在某个代码区域中加入硬编码的值和暴力逻辑，以绕过一个难以调试的 bug。
4. 开发者 A 说："写代码应当遵循规范。"开发者 B 答："我遵循了，看！"A 最终发现 B 所说的"规范"是他**前公司**的实践。A 于是把"我们"公司的规范发给 B，要求他改正。但 B 忙于其他编码工作，把修改这段代码措辞的工作**推迟到下一个开发周期**（例如 Scrum 的 Sprint）。
5. 我们**时间不够**了，所以这周人人都当程序员。Josh 停止校对文档，去帮 Mary 写代码。
6. 一次代码评审会上，一个开发者问："这里这个值是干什么用的？"另一个答："谁知道？它**在那儿好几年了！**系统跑得挺好。"追查历史后发现，曾有人试图从代码中删掉这个值，但一删就崩，没有人知道怎么解决。
7. Mary 能力很强，被要求改进某应用组件的错误处理。她有信心一周内完成。然而深入代码后，她发现逻辑不一致、糟糕的依赖和其他代码问题。于是她决定在改进错误处理**之前**先把这些问题全部清理掉。修改过程中她又发现**没有单元测试用例；**尽管如此，她仍继续对代码做重大修改，最终把代码提交进了主代码库。一切看起来正常。然而她没有意识到某些配置文件里**深度隐藏的依赖问题；**另一些代码段中为绕过历史 bug 而设的临时值所绑定的逻辑，被她的修改**弄断了，**导致应用失败。最终她把一切修好，但原本预估一周的工作量变成了三周。而且她的修改推送后，其他开发者**有时能测、有时不能测**（因为依赖被破坏）。所有开发者都很困惑：不知道如何在支撑应用交付和维护的同时修复这个烂摊子。

(b) 有一种说法是：累积的技术债务会慢慢拖垮一个项目乃至一家公司。许多人说技术债务不可避免；许多人也说项目在应用退役前并不会还清所有技术债务。请提出一个**成本有效**的软件项目技术债务应对策略。（8 分）

(c) 在你的课程项目中举一个技术债务的例子，须同时属于**文档类、有意、短期、不聚焦。**（3 分）

解答 / Answers

(a) Four-dimension classification (14 pts, 2 pts each)

English answer:

#	Item (summary)	Type	Intent.	Time	Focus	Reasoning
1	Skip blockchain support for the next two releases, but record considerations in a folder	**D**	**I**	**L**	**F**	A single, deliberate architectural/design decision (not preparing the decentralized-app structure), made strategically for two releases — yet it is one identifiable decision, hence focused.
2	Changed a code file, forgot to update unit tests and to pass them to buddies for review	**T**	**U**	**S**	**F**	Test debt: test cases were simply forgotten (no deliberate decision) while making one change — an identifiable, short-term oversight.
3	Hard-coded values + brute-force logic to bypass a hard-to-debug bug	**C**	**I**	**S**	**F**	A deliberate quick-and-dirty workaround in one code region: knowingly taking a shortcut now to save time.
4	Followed a "standard" from the former company; deferred fixing the wording to the next sprint	**C**	**U**	**S**	**F**	Unintentional (B believed he followed a standard), about one specific piece of code, deferred only to the next sprint.

#	Item (summary)	Type	Intent.	Time	Focus	Reasoning
5	Out of time: stop proofreading documentation, everyone codes this week	**O**	**I**	**S**	**F**	Documentation debt deliberately incurred to meet the deadline, concerning the documentation task at hand.
6	Mysterious value in code for years; removing it always breaks things	**C**	**U**	**L**	**F**	A single code artifact whose purpose nobody knows — unintentional, long-lived ("there for years"), yet one identifiable item.
7	Mary's broad cleanup: no unit tests, hidden dependency/config couplings broken, everyone puzzled	**D** (also T)	**I**	**S**	**U**	She deliberately chose to make major changes without tests; the damage comes from widespread bad dependencies/couplings across code and config — many scattered issues rather than one shortcut, hence unfocused; the impact unfolded within a single task (one week became three).

Note: for item 7, if only one primary type is allowed, answer **D (design/architecture debt)** — what actually breaks the system is the bad dependencies and hidden couplings; the missing unit tests are accompanying test debt.

中文对照：

#	条目（概括）	类型	意图性	时限	聚焦度	理由
1	放弃两个版本内的区块链支持，但把考虑记入文件夹	**D**	**I**	**L**	**F**	有意做出的**单一架构/设计决策**（不为去中心化应用做准备），战略层面跨越两个发布版本；但债务点本身可单独识别，故聚焦。
2	改了代码文件，忘记更新单元测试，也忘记送伙伴评审	**T**	**U**	**S**	**F**	测试债务：改代码时**单纯忘记**更新测试用例（并非有意决定），属于一次具体变更产生的、可识别的短期疏忽。
3	加硬编码值和暴力逻辑绕过难调试的 bug	**C**	**I**	**S**	**F**	明知是权宜之计仍在一个代码区域内硬编码绕过，典型的聚焦短期代码债务。
4	沿用了前公司的"规范"，推迟到下个 Sprint 再改	**C**	**U**	**S**	**F**	**无意**之中沿用了前公司的规范（B 自以为遵循了规范），针对具体代码，只推迟一个 Sprint。
5	时间不够：停止校对文档，这周全员写代码	**O**	**I**	**S**	**F**	为赶截止日期**有意**暂停文档校对的文档债务，针对手头的文档工作。
6	代码里一个存在多年、无人知晓用途的值	**C**	**U**	**L**	**F**	单一代码制品，无人知晓用途——无意产生、存在多年（长期），但债务点单一可识别。
7	Mary 大范围清理：无单元测试、隐藏依赖被破坏、全员困惑	**D**（兼 T）	**I**	**S**	**U**	她**有意**在缺乏测试的情况下大改代码；真正拖垮系统的是遍布代码与配置文件的糟糕依赖与隐藏耦合——无数分散问题而非单点捷径，故不聚焦；影响在一个任务周期内爆发（一周变三周）。（缺少单元测试是伴随的测试债务。）

(b) A cost-effective strategy to handle technical debt (8 pts)

English answer:

Since technical debt is unavoidable and a project never repays all of it before retirement, the strategy should be **"make it visible, manage it by value and risk, and repay it continuously in small, planned doses"**:

1. **Make TD visible and tracked (audit + backlog).** Run regular TD audits with the team; record each debt item — location, description, and the potential cost of not fixing it — explicitly in the product backlog or task board instead of leaving it in shadow backlogs. Measure debt in money (e.g., "% of the dev budget spent on non-productive maintenance"), not in vague feelings.

2. **Prioritize by cost/value (risk management).** Not all debt deserves repayment: debt that "will never be seen by the customer" can be left alone ("if it ain't broke, don't fix it"), while high-interest debt (brittle code that breaks on every change, like items 6–7 in (a)) is ranked first. Allocate a fixed share of each release cycle — e.g., 5–10% of every sprint — to repaying the prioritized debt, so repayment is steady and predictable rather than a crisis.
3. **Repay by refactoring, supported by cheap tools.** Refactoring is the only significantly effective way to reduce TD; use static analyzers/linters (best practice, minimal overhead) to detect debt early, reserve regular time slots for refactoring, and prioritize critical areas first with the Product Owner involved.
4. **Align business and technology.** Manage customer/stakeholder expectations through open dialogue (both sides as equal partners): educate developers on the business context and management on the cost of debt, so repayment effort is an accepted part of every plan instead of hidden overtime.

This is cost-effective because effort is spent only on debt whose interest exceeds its repayment cost, in small continuous doses that avoid both the 40–50% maintenance trap and the waste of repaying debt that will never matter.

中文对照： 既然技术债务不可避免、项目退休前也不可能全部还清，策略应是**"让债务可见 → 按价值和风险排序 → 小剂量持续偿还"：**

1. **让债务可见、可跟踪（审计 + 待办列表）。** 定期与团队做技术债务审计，把每项债务（位置、描述、不还的潜在代价）明确记入产品待办列表/任务板，而不是留在影子待办里；用钱衡量债务（如"研发预算的百分之几花在非生产性维护上"）。
2. **按成本/价值优先排序（风险管理）。**
   并非所有债都值得还：客户永远看不到的债可以"没坏就不修"；而高息债务（像 (a) 中第 6、7 条那种一改就碎的脆弱代码）排最前。每个发布周期固定划出一定比例（如每个 Sprint 的 5–10%）还债，使偿还可预测、可持续。
3. **用重构还债，辅以低成本工具。** 重构是减少技术债务唯一显著有效的方法；用静态分析器/lint（开销最小、最佳实践）早发现债务，定期预留重构时间段，优先处理关键区域，并让产品负责人参与。
4. **业务与技术对齐。** 通过开放对话管理客户和利益相关者的期望（双方作为平等伙伴）：让开发者理解业务背景、让管理层理解债务的代价，使还债成为每个计划中被认可的部分，而不是隐性加班。

之所以"成本有效"，是因为只把精力花在利息超过偿还成本的债务上，并以小剂量持续投入，既避免陷入 40–50% 维护时间的陷阱，也不浪费在永远无关紧要的债务上。

(c) Example: Documentation + Intentional + Short-term + Unfocused (3 pts)

English answer:

"In our course project, one week before the milestone demo we deliberately stopped writing and updating the README, the API documentation and the inline comments for the newly added modules, so that all team members could concentrate on finishing the features on time. We planned to complete the documentation right after the demo."

- **Documentation** — the missing README, API docs and comments are documentation debt.
- **Intentional** — the team knowingly made this trade-off to hit the deadline.
- **Short-term** — it was a reaction to an immediate schedule crunch, to be fixed right after the demo (one sprint at most).
- **Unfocused** — the debt is scattered across many modules and files, not a single identifiable document or section.

中文对照： "在课程项目中，里程碑演示前一周，我们有意暂停编写和更新新增模块的 README、API 文档和注释，好让全队集中精力按时完成功能，打算演示后再补文档。"

- **文档债务**——缺的是 README、API 文档和注释；
- **有意**——团队明知而为，为赶截止日期主动取舍；

- **短期**——是对眼前时间压力的即时反应，计划演示后（至多一个 Sprint）就补；
- **不聚焦**——债务分散在许多模块和文件中，不是某一份可单独指出的文档。

附：题目考点与课程对应关系 / Topic-to-Week Mapping

题目	考点	对应课程
Q1 软件过程	敏捷 vs 传统过程（迭代、客户参与、优先级、短周期、自组织团队）	Week 2 过程模型（XP/Scrum）
Q2 软件需求	可测量的非功能需求；需求获取五活动	Week 4 需求工程
Q3 软件架构	质量属性提取与转述；MVC 适用性；面向特殊用户的设计	Week 5 架构与设计
Q4 技术债务	四维度分类（Type/Intentionality/Time horizon/Focus）；债务管理策略	Week 6 技术债务